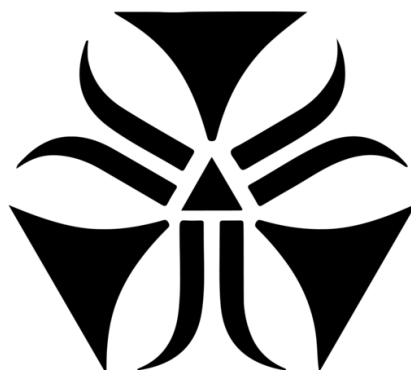




دانشگاه صنعتی همدان

World cup simulation 2026

نام و نام خانوادگی دانشجو

مأده شاه محمدی

استاد راهنما

مرتضی بهرامی

تابستان 1405

مقدمه و اهداف پروژه :

در این پروژه یک شبیه‌ساز جام جهانی فوتبال 2026 با رویکرد کلاسیک 32 تیمی و با استفاده از برنامه‌نویسی شیء‌گرا در زبان پایتون طراحی و پیاده‌سازی می‌خواند، گروه‌بندی را CSV شده است. برنامه اطلاعات تیم‌ها را از یک فایل بر اساس سیدبندی رنکینگ فیفا انجام می‌دهد، مرحله گروهی و تمام مراحل حذفی (یک‌هشتم، یک‌چهارم، نیمه‌نهایی و فینال) را شبیه‌سازی می‌کند، قهرمان را مشخص می‌کند و امکان اجرای هزار باره تورنمنت برای گزارش درصد قهرمانی هر تیم را نیز فراهم می‌کند.

نحوه شبیه سازی :

تعداد گل هر تیم در هر مسابقه بر اساس توزیع پواسون و با میانگین (لامبدا) محاسبه شده از قدرت حمله تیم و قدرت دفاع حریف تولید می‌شود. در مرحله گروهی، نتیجه مساوی بدون وقت اضافه باقی می‌ماند. در مراحل حذفی، در صورت تساوی پس از 90 دقیقه، یک وقت اضافه 30 دقیقه‌ای با میانگین گل کاهش یافته شبیه‌سازی می‌شود و در صورت تداوم تساوی، نتیجه با ضربات پنالتی (پنج ضربه برای هر تیم و در صورت لزوم پنالتی ناگهانی) مشخص می‌شود. احتمال گل شدن هر ضربه پنالتی بر اساس قدرت حمله زننده و قدرت دفاع حریف محاسبه و در بازه [۰.۶ , ۰.۹] محدود می‌شود.

معماری و شرح کلاس ها :

متد ها	نقش کلاس	Class
<code>simulate_match()</code> , <code>lambda_vs()</code> <code>_simulate_penalties()</code> <code>goal_difference()</code> , <code>reset_stats()</code> <code>penalty_prob_vs()</code>	مدل سازی تیم ها قدرت و امار	Team
<code>play()</code> <code>result_str()</code>	مدريت مستقيم دو تیم	Match
<code>matches_all_play()</code> , <code>ranking_get()</code> <code>advance_teams()</code> , <code>table_display()</code>	رسم جدول و مدريت بازی بين 4 تیم	Group
<code>round_play()</code> , <code>winners_get()</code> <code>results_display()</code> ,	مدريت مراحل حذفی و صعودی	KnockoutStage
<code>load_teams_form_csv()</code> <code>groups_draw_and_seed()</code> <code>champion_likegy_most()</code> <code>stage_group_run()</code> <code>bracket_knockout_setup()</code> <code>stage_knockout_run()</code> , <code>simulation_full_run()</code> <code>bracket_display()</code>	موتور اصلی	WorldCupSimulator

Team

نگهداری اطلاعات پایه هر تیم ملی (نام، قدرت حمله، قدرت دفاع، رتبه فیفا) و آمار آن در طول تورنمنت (گل زده، گل خورده، امتیاز). منطق اصلی شبیه‌سازی یک مسابقه، شامل محاسبه گل بر اساس توزیع پواسون، وقت اضافه و ضربات پنالتی، در این کلاس پیاده‌سازی شده است

```
[7]: team_a = Team("Argentina", 90, 80, 2)
     team_b = Team("Germany", 85, 85, 7)

     print(f"1 تیم: {team_a.name}, حمله: {team_a.attack}, دفاع: {team_a.defense}, رتبه: {team_a.rank}")
     print(f"امتیاز (اولیه): {team_a.against_goals}, گل های خورده (اولیه): {team_a.for_goals}, امتیاز (اولیه): 0")
     print(f"2 تیم: {team_b.name}, حمله: {team_b.attack}, دفاع: {team_b.defense}, رتبه: {team_b.rank}")

     حمله: 90, دفاع: 80, رتبه: 2, تیم: Argentina
     گل های زده (اولیه): 0, گل های خورده (اولیه): 0, امتیاز (اولیه): 0
     حمله: 85, دفاع: 85, رتبه: 7, تیم: Germany

[8]: team_a.for_goals = 3
     team_a.against_goals = 1

     print(f"تفاضل گل {team_a.name}: {team_a.goal_difference()}")

     Argentina: 2 تفاضل گل

[9]: print(f"امتیاز {team_a.against_goals}, گل خورده {team_a.for_goals}, قبل از بازنشانی: گل زده {team_a.name} آمار {team_a.points}")
     team_a.reset_stats()
     print(f"امتیاز {team_a.against_goals}, گل خورده {team_a.for_goals}, بعد از بازنشانی: گل زده {team_a.name} آمار {team_a.points}")

     قبل از بازنشانی: گل زده 3, گل خورده 1, امتیاز 0 Argentina آمار
     بعد از بازنشانی: گل زده 0, گل خورده 0, امتیاز 0 Argentina آمار

[10]: lambda_val_a = team_a.lambda_vs(team_b)
      lambda_val_b = team_b.lambda_vs(team_a)

      print(f"مقابل {team_b.name}: {lambda_val_a:.2f} لامبدا (گل مورد انتظار) {team_a.name}")
      print(f"مقابل {team_a.name}: {lambda_val_b:.2f} لامبدا (گل مورد انتظار) {team_b.name}")

      Germany مقابل Argentina لامبدا (گل مورد انتظار) 1.47
      Argentina مقابل Germany لامبدا (گل مورد انتظار) 1.43
```

Match

مسئول اجرای یک مسابقه مشخص بین دو تیم، به روزرسانی آمار تیم‌ها بر اساس نتیجه، امتیازدهی در مرحله گروهی و تعیین برنده در مراحل حذفی.

```
[4]: team_iran = Team("Iran", 75, 78, 20)
     team_usa = Team("USA", 70, 72, 28)

[5]: match_group = Match(team_iran, team_usa, is_knockout=False)
     print(f"مسابقه گروهی: {match_group.team1.name} vs {match_group.team2.name}, حذفی: {match_group.is_knockout}")
     مسابقه گروهی: Iran vs USA, حذفی: False

[6]: match_knockout = Match(team_usa, team_iran, is_knockout=True)
     print(f"مسابقه حذفی: {match_knockout.team1.name} vs {match_knockout.team2.name}, حذفی: {match_knockout.is_knockout}")
     مسابقه حذفی: USA vs Iran, حذفی: True

[7]: ##play بررسی متد##

[8]: print(f"امتیاز {team_iran.name} گل زده: {team_iran.for_goals}, گل خورده: {team_iran.against_goals}, امتیاز {team_iran.points}")
     print(f"امتیاز {team_usa.name} گل زده: {team_usa.for_goals}, گل خورده: {team_usa.against_goals}, امتیاز {team_usa.points}")
     گل از بازی: گل زده 0, گل خورده 0, امتیاز 0 Iran
     گل از بازی: گل زده 0, گل خورده 0, امتیاز 0 USA

[9]: match_group.play()

[10]: print(f"نتیجه بازی: {match_group.goals1}-{match_group.goals2}")
      print(f"امتیاز {team_iran.name} بعد از بازی: گل زده {team_iran.for_goals}, گل خورده: {team_iran.against_goals}, امتیاز {team_iran.points}")
      print(f"امتیاز {team_usa.name} بعد از بازی: گل زده {team_usa.for_goals}, گل خورده: {team_usa.against_goals}, امتیاز {team_usa.points}")

نتیجه بازی: 1-0
بعد از بازی: گل زده 0, گل خورده 1, امتیاز 0 Iran
بعد از بازی: گل زده 1, گل خورده 0, امتیاز 3 USA

[11]: # برای بازی حذفی، تیم‌ها را ریست می‌کنیم یا نمونه جدید می‌سازیم
     team_iran_ko = Team("Iran", 75, 78, 20)
     team_usa_ko = Team("USA", 70, 72, 28)
     match_knockout_example = Match(team_iran_ko, team_usa_ko, is_knockout=True)

[12]: print(f"امتیاز {team_iran_ko.name} گل زده: {team_iran_ko.for_goals}, گل خورده: {team_iran_ko.against_goals}, امتیاز {team_iran_ko.points}")
     match_knockout_example.play()
     گل از بازی: گل زده 0, گل خورده 0, امتیاز 0 Iran

[13]: # بعد از بازی حذفی
     print(f"نتیجه بازی: {match_knockout_example.goals1}-{match_knockout_example.goals2}")
     if match_knockout_example.winner:
         print(f"برنده: {match_knockout_example.winner.name}")
     else:
         print("برنده مشخص نشد (نباید در حذفی رخ دهد)")
     print(f"امتیاز {team_iran_ko.name} بعد از بازی: گل زده {team_iran_ko.for_goals}, گل خورده: {team_iran_ko.against_goals}, امتیاز {team_iran_ko.points}")

نتیجه بازی: 0-1
برنده: Iran
بعد از بازی: گل زده 1, گل خورده 0, امتیاز 0 Iran
```

Group

نگهداری چهار تیم یک گروه، اجرای تمام مسابقات درون‌گروهی و رتبه‌بندی نهایی تیم‌ها بر اساس امتیاز، تفاضل گل، گل زده و در نهایت قرعه‌کشی تصادفی در صورت تساوی کامل.

```
[6]: team1 = Team("Qatar", 65, 60, 50)
    team2 = Team("Ecuador", 75, 74, 44)
    team3 = Team("Senegal", 80, 78, 18)
    team4 = Team("Netherlands", 88, 85, 8)

[7]: group_a = Group("A", [team1, team2, team3, team4])
    print(f"گروه ایجاد شد: {group_a.name} با {len(group_a.teams)} تیم")
    با 4 تیم A: گروه ایجاد شد

[8]: played_matches = group_a.matches_all_play()
    print(f"تعداد مسابقات انجام‌شده در گروه {group_a.name}: {len(played_matches)}")
    for match in played_matches:
        print(match.result_str())

    A: 6 تعداد مسابقات انجام‌شده در گروه
    Qatar 1-2 Ecuador -> برنده: Ecuador
    Qatar 1-1 Senegal
    Qatar 0-0 Netherlands
    Ecuador 2-3 Senegal -> برنده: Senegal
    Ecuador 2-4 Netherlands -> برنده: Netherlands
    Senegal 2-1 Netherlands -> برنده: Senegal

[10]: ranked_teams = group_a.ranking_get()
    print(f"امتیاز {ranked_teams[0].points} با {ranked_teams[0].name} تیم اول گروه")
    با 16 امتیاز Senegal: تیم اول گروه

[11]: first_team, second_team = group_a.advance_teams()
    print(f"تیم دوم {second_team.name} و {first_team.name} (تیم اول) از گروه {group_a.name} تیم‌های صعودکننده از گروه")
    (تیم دوم) Netherlands و (تیم اول) Senegal A: تیم‌های صعودکننده از گروه

[12]: print(group_a.table_display())

===== Group A =====
1. Senegal: 16 pts, GD +5, GF 9
2. Netherlands: 8 pts, GD +1, GF 9
3. Qatar: 5 pts, GD -2, GF 5
4. Ecuador: 4 pts, GD -4, GF 7
```

KnockoutStage

نگهداری و اجرای مسابقات یک مرحله حذفی مشخص (مثلا یک هشتم نهایی) و برگرداندن لیست تیم‌های برنده برای ساخت مرحله بعد.

```
[6]: team_a = Team("Argentina", 90, 85, 2)
     team_b = Team("Netherlands", 85, 80, 8)
     team_c = Team("Brazil", 92, 88, 1)
     team_d = Team("Croatia", 80, 82, 12)

[7]: matchups = [(team_a, team_b), (team_c, team_d)]
     round16 = KnockoutStage("Round of 16", matchups)

[8]: round16.round_play()

[9]: winners = round16.winners_get()
     print(f"تعداد تیم‌های صعودکننده: {len(winners)}")
     تعداد تیم‌های صعودکننده: 2

[10]: print(round16.results_display())

===== Round of 16 =====
Argentina 2-1 Netherlands -> برنده: Argentina
Brazil 4-2 Croatia -> برنده: Brazil
```

WorldCupSimulator

کلاس اصلی و هماهنگ‌کننده کل فرآیند تورنمنت؛ شامل خواندن فایل (Csv) قرعه‌کشی گروه‌ها، اجرای مرحله گروهی، ساخت براکت حذفی طبق قانون فیفا، اجرای مراحل حذفی و اجرای چندباره تورنمنت برای محاسبه درصد قهرمانی هر تیم.

```
[14]: simulator = WorldCupSimulator()

[15]: success = simulator.load_teams_from_csv("t1.csv")
if success:
    print(f"تعداد تیم‌های بارگذاری شده: {len(simulator.teams)}")
    print(f"اولین تیم: {simulator.teams[0].name}، رنگ: {simulator.teams[0].rank}")
else:
    print("بارگذاری تیم‌ها با شکست مواجه شد")

بارگذاری شد 't1.csv' تیم با موفقیت از فایل 32
تعداد تیم‌های بارگذاری شده: 32
رنگ: 1: Argentina, اولین تیم

[16]: success_draw = simulator.groups_draw_and_seed()
if success_draw:
    print(f"تعداد گروه‌های ایجاد شده: {len(simulator.groups)}")
    for i, group in enumerate(simulator.groups):
        print(f"گروه {group.name}: {[team.name for team in group.teams]}")
else:
    print("قرعه‌کشی گروه‌ها با شکست مواجه شد")

قرعه‌کشی گروه‌ها با موفقیت انجام شد.
تعداد گروه‌های ایجاد شده: 8
گروه A: ['Portugal', 'Colombia', 'Iran', 'Poland']
گروه B: ['England', 'USA', 'Switzerland', 'Ghana']
گروه C: ['Argentina', 'Uruguay', 'Ecuador', 'Saudi Arabia']
گروه D: ['Brazil', 'Mexico', 'Japan', 'Canada']
گروه E: ['France', 'Morocco', 'South Korea', 'Serbia']
گروه F: ['Netherlands', 'Croatia', 'Australia', 'Tunisia']
گروه G: ['Belgium', 'Italy', 'Senegal', 'Nigeria']
گروه H: ['Spain', 'Germany', 'Denmark', 'Wales']

[17]: success_group_stage = simulator.stage_group_run()
if success_group_stage:
    print("\nموفقیت به پایان رسید.")
else:
    print("\nاجرای مرحله گروهی با شکست مواجه شد.")

===== Group A =====
1. Colombia: 7 pts, GD +3, GF 5
2. Poland: 4 pts, GD +1, GF 4
3. Iran: 3 pts, GD -2, GF 3
4. Portugal: 3 pts, GD -2, GF 2
===== Group B =====
1. USA: 9 pts, GD +8, GF 11
2. Switzerland: 6 pts, GD +1, GF 4
3. Ghana: 3 pts, GD -3, GF 2
4. England: 0 pts, GD -6, GF 2
===== Group C =====
1. Uruguay: 7 pts, GD +4, GF 7
2. Argentina: 7 pts, GD +3, GF 4
3. Saudi Arabia: 1 pts, GD -2, GF 1
4. Ecuador: 1 pts, GD -5, GF 3
===== Group D =====
1. Brazil: 5 pts, GD +1, GF 3
2. Mexico: 4 pts, GD +0, GF 4
3. Japan: 4 pts, GD +0, GF 4
4. Canada: 2 pts, GD -1, GF 4
===== Group E =====
1. France: 7 pts, GD +2, GF 5
2. Morocco: 6 pts, GD +6, GF 8
3. South Korea: 2 pts, GD -3, GF 3
4. Serbia: 1 pts, GD -5, GF 1
```

```

===== Group F =====
1. Netherlands: 7 pts, GD +2, GF 5
2. Croatia: 5 pts, GD +1, GF 5
3. Australia: 4 pts, GD +0, GF 2
4. Tunisia: 0 pts, GD -3, GF 4
===== Group G =====
1. Italy: 7 pts, GD +3, GF 5
2. Senegal: 4 pts, GD +0, GF 5
3. Belgium: 4 pts, GD +0, GF 2
4. Nigeria: 1 pts, GD -3, GF 2
===== Group H =====
1. Spain: 7 pts, GD +7, GF 7
2. Wales: 5 pts, GD +1, GF 5
3. Denmark: 2 pts, GD -3, GF 2
4. Germany: 1 pts, GD -5, GF 2

مرحله گروهی با موفقیت به پایان رسید.

[18]: success_bracket_setup = simulator.bracket_knockout_setup()
if success_bracket_setup:
    print("براکت یک‌هشتم نهایی با موفقیت تنظیم شد.")
    print(f"تعداد بازی‌ها در مرحله یک‌هشتم: {len(simulator.round_of_16.matches)}")
    print(f"اولین بازی: {simulator.round_of_16.matches[0].team1.name} vs {simulator.round_of_16.matches[0].team2.name}")
else:
    print("تنظیم براکت یک‌هشتم نهایی با شکست مواجه شد.")

براکت یک‌هشتم نهایی با موفقیت تنظیم شد.
تعداد بازی‌ها در مرحله یک‌هشتم: 8
اولین بازی: Colombia vs Switzerland

[19]: champion_team = simulator.stage_knockout_run()
if champion_team:
    print(f"\n🏆 قهرمان جام جهانی: {champion_team.name}")
else:
    print("\nتینین قهرمان با شکست مواجه شد.")

🏆 قهرمان جام جهانی: Croatia

[20]: full_simulator = WorldCupSimulator()

[21]: full_simulator.load_teams_from_csv("t1.csv")

بارگذاری شد 't1.csv' تیم با موفقیت از فایل 32.

[21]: True

[22]: single_run_champion = full_simulator.simulation_full_run()
if single_run_champion:
    print(f"\nقهرمان یک اجرای کامل شبیه‌سازی: {single_run_champion.name}")
else:
    print("\nشبیه‌سازی کامل با شکست مواجه شد و قهرمانی تعیین نشد.")

قرعه‌کشی گروه‌ها با موفقیت انجام شد.

قهرمان یک اجرای کامل شبیه‌سازی: Netherlands

[27]: probabilities = full_simulator.champion_likely_most(simulations_num=5)

قرعه‌کشی گروه‌ها با موفقیت انجام شد.
قرعه‌کشی گروه‌ها با موفقیت انجام شد.
قرعه‌کشی گروه‌ها با موفقیت انجام شد.
قرعه‌کشی گروه‌ها با موفقیت انجام شد.
قرعه‌کشی گروه‌ها با موفقیت انجام شد.
شبیه‌سازی 5 بار انجام شد.
درصد قهرمانی هر تیم:
Brazil: 40.0%
Spain: 20.0%
South Korea: 20.0%
Australia: 20.0%

```

چالش های که در طی پروژه داشتم :

چالش مدل سازی واقع گرایانه گل ها بدون وابستگی به (Numpy)

راه حل : (knuth) استفاده از الگوریتم

چالش ایزوله سازی آمار در شبیه سازیهای هزارگانه

راه حل : استفاده از تابع ریست

جمع بندی :

پروژه با موفقیت تمام نیازمندی های اجباری صورت پروژه شامل خواندن تیم ها از فایل وسیدبندی گروه ها، شبیه سازی مرحله گروهی و تمام مراحل حذفی با رعایت وقت اضافه و پناالتی، تعیین قهرمان، اجرای هزار باره تورنمنت و گزارش درصد قهرمانی، و نمایش متنی براکت حذفی را پیاده سازی کرده است .